Practical-2: Study and implement various operations on Python lists such as creation, indexing, slicing, modification, traversal, sorting, nested lists, and list comprehensions.

Create a Python list marks to represent the marks obtained by students of a class and perform the following tasks:

1. Populate the list marks with the marks of at least 10 students.

```
marks = [85, 90, 78, 92, 88, 76, 95, 82, 89, 91]
```

2. Display the complete list.

```
print("Complete list:", marks)
```

```
Complete list: [85, 90, 78, 92, 88, 76, 95, 82, 89, 91]
```

3. Display the first and last elements of the list.

```
print("First element:", marks[0])
print("Last element:", marks[-1])
```

```
First element: 85
Last element: 91
```

4. Display marks from index 2 to index 6 using slicing.

```
print("Marks from index 2 to 6:", marks[2:7])
```

```
Marks from index 2 to 6: [78, 92, 88, 76, 95]
```

5. Find and display the total number of students using len().

```
print("Total number of students:", len(marks))
```

```
Total number of students: 10
```

6. Update the marks of the third student.

```
marks[2] = 80
print("Updated list:", marks)
```

```
Updated list: [85, 90, 80, 92, 88, 76, 95, 82, 89, 91]
```

7. Append the marks of a new student to the list.

```
marks.append(85)
print("List after appending:", marks)
```

```
List after appending: [85, 90, 80, 92, 88, 76, 95, 82, 89, 91, 85]
```

8. Insert the marks of a student at a specified position.

```
marks.insert(3, 90)
print("List after inserting:", marks)
```

```
List after inserting: [85, 90, 80, 90, 92, 88, 76, 95, 82, 89, 91, 85]
```

9. Remove a specific student's marks using remove().

```
marks.remove(85)
print("List after removing:", marks)
```

```
List after removing: [90, 80, 90, 92, 88, 76, 95, 82, 89, 91, 85]
```

10. Remove the last element using pop().

```
marks.pop()
print("List after popping:", marks)
```

```
List after popping: [90, 80, 90, 92, 88, 76, 95, 82, 89, 91]
```

11. Traverse the list using a for loop.

```
print("Traversing using for loop:")
for mark in marks:
    print(mark)
```

```
Traversing using for loop:
90
80
90
92
88
76
95
82
89
91
```

12. Traverse the list using a while loop.

```
print("Traversing using while loop:")
i = 0
while i < len(marks):
    print(marks[i])
    i += 1
```

```
Traversing using while loop:
90
80
90
92
88
76
95
82
89
91
```

13. Search for a given mark and display its index position.

```
search_mark = 82
if search_mark in marks:
    print("Mark", search_mark, "found at index", marks.index(search_mark))
else:
    print("Mark", search_mark, "not found")
```

```
Mark 82 found at index 7
```

14. Count and display the number of students who scored more than 75 marks.

```
count = 0
for mark in marks:
    if mark > 75:
        count += 1
print("Number of students who scored more than 75 marks:", count)
```

```
Number of students who scored more than 75 marks: 10
```

15. Sort the list in ascending order.

```
marks.sort()
print("Sorted list:", marks)
```

```
Sorted list: [76, 80, 82, 88, 89, 90, 90, 91, 92, 95]
```

16. Sort the list in descending order.

```
marks.sort(reverse=True)
print("Sorted list in descending order:", marks)
```

```
Sorted list in descending order: [95, 92, 91, 90, 90, 89, 88, 82, 80, 76]
```

17. Reverse the list without sorting.

```
marks.reverse()
print("Reversed list:", marks)
```

```
Reversed list: [76, 80, 82, 88, 89, 90, 90, 91, 92, 95]
```

18. Find and display the maximum and minimum marks.

```
print("Maximum mark:", max(marks))
print("Minimum mark:", min(marks))
```

```
Maximum mark: 95
Minimum mark: 76
```

19. Find and display the sum and average of marks.

```
print("Sum of marks:", sum(marks))
print("Average mark:", sum(marks) / len(marks))
```

```
Sum of marks: 873
Average mark: 87.3
```

20. Create a copy of the list using slicing.

```
marks_copy = marks[:]  # or marks.copy()
print("Copy of the list:", marks_copy)
```

```
Copy of the list: [76, 80, 82, 88, 89, 90, 90, 91, 92, 95]
```

21. Create another copy using the list() function.

```
marks_copy2 = list(marks)
print("Another copy of the list:", marks_copy2)
```

```
Another copy of the list: [76, 80, 82, 88, 89, 90, 90, 91, 92, 95]
```

22. Compare assignment and copying of lists.

```
print("\n--- Assignment vs Copying ---")
original_list = [1, 2, 3, 4, 5]
assigned_list = original_list  # Assignment - both point to the same object
copied_list = original_list[:]  # Copying - creates a new object

# Modify the original list
original_list[0] = 100

print("Original list after modification:", original_list)
print("Assigned list (affected by change):", assigned_list)  # Also changes
print("Copied list (not affected):", copied_list)  # Remains unchanged
print("Conclusion: Assignment creates a reference, copying creates an independent copy.")
```

```

--- Assignment vs Copying ---
Original list after modification: [100, 2, 3, 4, 5]
Assigned list (affected by change): [100, 2, 3, 4, 5]
Copied list (not affected): [1, 2, 3, 4, 5]
Conclusion: Assignment creates a reference, copying creates an independent copy.
```

23. Create a nested list to store [Roll No, Name, Marks].

```
print("\n--- Nested List ---")
students = [
    [1, "Alice", 85],
    [2, "Bob", 90],
    [3, "Charlie", 78],
    [4, "Diana", 92],
    [5, "Eve", 88]
]
print("Student data (Roll No, Name, Marks):")
for student in students:
    print(f"  Roll No: {student[0]}, Name: {student[1]}, Marks: {student[2]}")
```

```
--- Nested List ---
Student data (Roll No, Name, Marks):
  Roll No: 1, Name: Alice, Marks: 85
  Roll No: 2, Name: Bob, Marks: 90
  Roll No: 3, Name: Charlie, Marks: 78
  Roll No: 4, Name: Diana, Marks: 92
  Roll No: 5, Name: Eve, Marks: 88
```

24. Extract and display only the marks from the nested list.

```
print("\n--- Extracted Marks ---")
extracted_marks = [student[2] for student in students]
print("Marks extracted from nested list:", extracted_marks)
```

```
--- Extracted Marks ---
Marks extracted from nested list: [85, 90, 78, 92, 88]
```

25. Convert all marks into grades (A/B/C/D) and store them in a new list.

```
print("\n--- Grades Conversion ---")
def get_grade(mark):
    if mark >= 90:
        return 'A'
    elif mark >= 80:
        return 'B'
    elif mark >= 70:
        return 'C'
    elif mark >= 60:
        return 'D'
    else:
        return 'F'

grades = [get_grade(mark) for mark in extracted_marks]
print("Marks:", extracted_marks)
print("Grades:", grades)

# Display student names with their grades
print("\nStudent Grades:")
for student, grade in zip(students, grades):
    print(f"  {student[1]}: {grade}")
```

```
--- Grades Conversion ---
```

```
Marks: [85, 90, 78, 92, 88]
Grades: ['B', 'A', 'C', 'A', 'B']

Student Grades:
  Alice: B
  Bob: A
  Charlie: C
  Diana: A
  Eve: B
```